Revisiting Activation Outliers in Transformer Quantization: Reproduction, Extensions, and Deployment Tradeoffs

Pranav Kumar Kaliaperumal
M.S. Computer Science
University of Colorado Denver
Aurora, Colorado, USA
pranavkumar.kaliaperumal@ucdenver.edu

***Abstract*—Post-training quantization (PTQ) enables efficient transformer inference, yet naive activation quantization frequently causes catastrophic accuracy collapse. Prior work attributes this degradation to structured activation outliers that persist and amplify across residual pathways.**

In this paper, we reproduce activation outlier–induced failure in BERT-base under W8A8 PTQ, analyze its statistical origin, and systematically evaluate mitigation strategies including mixed precision PTQ and per-embedding-group (PEG) quantization. We introduce a lightweight percentile-based calibration extension that reduces outlier dominance without sacrificing integer-only deployment simplicity.

Beyond accuracy, we perform deployment-aware profiling including latency (p50/p95), peak VRAM, and serialized model size on RTX 3050 hardware. All experiments are reproducible via a fully automated pipeline.

Our results clarify when full INT8 activation quantization fails, quantify tradeoffs between accuracy and deployment efficiency, and provide practical guidance for edge transformer inference.

I. Introduction

Transformer architectures now dominate natural language processing, yet their deployment remains constrained by computational and memory costs. While post-training quantization (PTQ) offers a practical compression pathway without retraining, transformer models exhibit a distinct failure mode under naive activation quantization. In particular, full W8A8 quantization frequently causes disproportionate accuracy degradation compared to convolutional networks.

Recent studies suggest that this degradation is not due to random quantization noise, but rather to *structured activation outliers* that emerge and persist across residual pathways. These outliers inflate activation ranges, causing uniform min-max scaling to allocate disproportionate dynamic range to a small subset of dimensions. As a result, the majority of activations collapse into narrow integer bins, increasing quantization error.

Despite growing awareness of this issue, several open questions remain:

- How consistently do activation outliers reproduce across training seeds and deployment conditions?
- To what extent does residual amplification drive quantization instability?
- How do mitigation strategies compare under real deployment constraints (latency, VRAM, model size)?
- Can a simpler calibration strategy recover performance without increasing runtime complexity?

In this work, we provide a reproducible systems-level investigation of activation outlier–induced PTQ failure in BERT-base on QNLI. We reproduce collapse under naive W8A8 quantization, analyze activation statistics across depth, and evaluate three mitigation strategies:

1) Mixed precision PTQ,
2) Per-Embedding-Group (PEG) quantization with permutation,
3) A percentile-based activation calibration extension (proposed).

Unlike prior work that focuses primarily on accuracy restoration, we evaluate each method across a deployment-aware axis including p50/p95 latency, peak VRAM usage, and serialized model size on RTX 3050 hardware.

Our key findings are:

- Structured activation outliers persist across residual pathways and amplify with depth.
- Naive W8A8 PTQ collapses under uniform scaling.
- PEG restores accuracy but introduces modest complexity overhead.
- Mixed precision is robust but reduces compression gains.
- Percentile-based calibration provides a strong middle ground, recovering substantial accuracy without increasing runtime cost.

All experiments are fully reproducible through an automated pipeline that executes training, quantization, profiling, and paper artifact generation in a single command.

This work clarifies the statistical and systems-level mechanisms behind transformer PTQ failure and provides practical guidance for deployment under resource constraints.

II. Problem Formulation

Let $x \in \mathbb{R}$ denote activation values drawn from distribution $\mathcal{D}_l$ at transformer layer l. Uniform affine quantization maps x to integer representation q:

$$q = \text{round}\left(\frac{x}{s}\right) + z \tag{1}$$

where s is the quantization scale, z the zero-point, and b the bit width (e.g., $b = 8$ for INT8).

The dequantized value is:

$$\hat{x} = s(q - z) \tag{2}$$

The quantization error is:

$$\epsilon(x) = x - \hat{x} \tag{3}$$

A. Min-Max Scaling Under Heavy-Tailed Distributions

Under symmetric min-max scaling,

$$s = \frac{\max(|x|)}{2^{b-1} - 1} \tag{4}$$

When activation distributions are heavy-tailed, i.e.,

$$P(|x| > t) \sim t^{-\alpha}, \quad \alpha < 4, \tag{5}$$

a small subset of extreme values dominates $\max(|x|)$.

Let δ denote the fraction of extreme outliers. Then the effective resolution allocated to the bulk of the distribution is reduced by a factor proportional to:

$$\rho = \frac{\sigma_{\text{bulk}}}{\max(|x|)} \tag{6}$$

As $\max(|x|)$ increases due to rare outliers, $\rho \to 0$, and the majority of activations collapse into narrow quantization bins.

B. Error Amplification in Residual Architectures

Transformer layers compute:

$$h_{l+1} = h_l + f_l(h_l) \tag{7}$$

Assume quantized representation:

$$\hat{h}_{l+1} = \hat{h}_l + \widehat{f_l(h_l)} \tag{8}$$

Let quantization error at layer l be $\epsilon_l = h_l - \hat{h}_l$.

Then error propagation satisfies:

$$\epsilon_{l+1} = \epsilon_l + \epsilon_{f_l} \tag{9}$$

Thus,

$$\epsilon_l = \sum_{k=0}^{l-1} \epsilon_{f_k} \tag{10}$$

If ϵ_{f_k} contains structured bias due to outlier scaling distortion, the error accumulates linearly with depth.

Moreover, variance grows as:

$$\text{Var}[\epsilon_l] = \sum_{k=0}^{l-1} \text{Var}[\epsilon_{f_k}] \tag{11}$$

Under uniform scaling distorted by heavy-tailed outliers, $\text{Var}[\epsilon_{f_k}]$ increases significantly relative to well-behaved CNN distributions.

C. Structured Channel Dominance

Empirically, we observe that activation magnitudes are not independently heavy-tailed across dimensions. Instead, specific embedding dimensions d_i consistently dominate variance:

$$\text{Var}(x_{d_i}) \gg \text{Var}(x_{d_j}) \tag{12}$$

for a small subset of i.

Let $\mathcal{I}$ denote top-$p\%$ dominant dimensions. Then:

$$\sum_{i \in \mathcal{I}} \text{Var}(x_{d_i}) \gg \sum_{j \notin \mathcal{I}} \text{Var}(x_{d_j}) \tag{13}$$

Uniform global scaling forces the entire activation tensor to accommodate these dominant channels, degrading resolution for the majority of dimensions.

D. Core Failure Mechanism

Combining these observations, transformer PTQ failure under naive W8A8 arises from:

1) Heavy-tailed activation distributions.
2) Persistent structured dominant channels.
3) Residual accumulation amplifying quantization bias.
4) Uniform global scaling misallocating dynamic range.

This motivates structured mitigation strategies that decouple dominant channels or reduce outlier influence during calibration.

III. Experimental Setup

A. Model and Dataset

We evaluate on BERT-base-uncased (110M parameters) fine-tuned on the QNLI task from the GLUE benchmark. QNLI is a binary sentence-pair classification task derived from question answering, containing:

- 104,743 training examples
- 5,463 validation examples

Accuracy on the validation set is used as the primary metric.

All experiments begin from a fine-tuned FP32 baseline to isolate quantization effects.

B. Training Configuration

Fine-tuning is performed with the following configuration:

- Optimizer: AdamW
- Learning rate: 5×10^{-5}
- Linear warmup schedule
- Batch size: 8
- Maximum sequence length: 128
- Weight decay: 0.01
- Gradient clipping: 1.0

- Number of epochs: consistent with GLUE baseline convergence

Training uses a fixed random seed (1000 unless otherwise specified). Deterministic CUDA behavior is enforced where applicable to ensure reproducibility.

Checkpoints are saved every 500 steps, and evaluation is performed at the same interval. Final model artifacts are stored under:

```
runs/fp32_seed1000/out/
```

C. Quantization Protocol

Post-training quantization (PTQ) is applied to the fine-tuned FP32 checkpoint. All PTQ variants share:

- Calibration on the training split
- Static activation range estimation
- Per-tensor affine quantization unless otherwise specified
- INT8 weight quantization for all methods

Activation calibration uses:

- Min-max scaling (W8A8 baseline)
- Layer-selective FP16 retention (Mixed Precision)
- Per-embedding-group scaling (PEG)
- Percentile-based range estimation (Proposed)

For percentile calibration, we compute:

$$s = \frac{\text{percentile}_{99.9}(|x|)}{127} \tag{14}$$

Calibration is performed in inference mode without gradient updates.

Quantized models are serialized to:

```
runs/<method>_seed1000/out/
```

D. Deployment Profiling Methodology

Deployment profiling is conducted under consistent runtime conditions:

- Hardware: NVIDIA RTX 3050 (6GB VRAM)
- CUDA 12.1
- PyTorch 2.2.2 with CUDA support
- Batch size: 8
- Sequence length: 128

Latency measurement procedure:

1) 100 warmup iterations (excluded)
2) 500 timed inference iterations
3) CUDA events for start/end timing
4) p50 and p95 latency computed

Peak VRAM usage is measured via CUDA memory tracking during inference. Model size is computed as the total size of serialized weight files, excluding checkpoint sprawl.

Deployment metrics are written to:

```
runs/results/deploy_profile.csv
```

E. Automation and Artifact Generation

All experiments are executed via a single entry point:

```
python oneclick/run_all.py
```

The pipeline:

1) Trains FP32 baseline (if not already present)
2) Applies each PTQ variant
3) Aggregates accuracy results
4) Profiles deployment metrics
5) Generates LaTeX-ready tables and figures

Result summaries are written to:

```
runs/results/method_metrics.csv
runs/results/deploy_profile.csv
paper/accuracy_table.tex
paper/deploy_table.tex
```

This design ensures exact reproducibility from raw training through paper artifact generation.

IV. Quantization Methods

We evaluate four post-training quantization (PTQ) strategies applied to a fine-tuned FP32 BERT-base model. All methods quantize weights to INT8. Activation quantization differs by method as described below.

Unless otherwise stated, we use static calibration with affine symmetric quantization and per-tensor activation scaling.

A. W8A8 Baseline (Uniform PTQ)

The baseline applies global min-max affine quantization to both weights and activations:

$$q = \text{round}\left(\frac{x}{s}\right) \tag{15}$$

with scale

$$s = \frac{\max(|x|)}{2^{b-1} - 1} \tag{16}$$

where $b = 8$.

Activation ranges are estimated using calibration data in inference mode. This approach treats all embedding dimensions and layers uniformly.

While simple and deployment-friendly, this method is highly sensitive to structured activation outliers.

B. Mixed Precision PTQ

Mixed precision retains selected layers in higher precision (FP16) while quantizing the remaining layers to INT8.

In our implementation, we retain FP16 precision for:

- Feed-forward network (FFN) output projections
- Residual summation inputs
- Attention output projections

All other linear layers use INT8 weights and activations.

This selectively protects layers most sensitive to activation range distortion while preserving partial compression benefits.

Mixed precision increases memory footprint relative to W8A8 but stabilizes accuracy.

C. Per-Embedding-Group Quantization (PEG)

To mitigate structured channel dominance, we partition embedding dimensions into K groups (default $K = 3$). Each group receives an independent activation scale:

$$q_g = \text{round}\left(\frac{x_g}{s_g}\right) \tag{17}$$

where

$$s_g = \frac{\max(|x_g|)}{2^{b-1} - 1} \tag{18}$$

This decouples dominant embedding dimensions from the bulk distribution.

1) Permutation Strategy: We apply a range-based permutation that reorders embedding dimensions prior to grouping. Dimensions are sorted by activation magnitude statistics and distributed across groups to prevent concentration of outliers within a single group.

Permutation is inverted after quantization to preserve functional equivalence.

PEG increases minor bookkeeping overhead but preserves integer arithmetic at runtime.

D. Percentile-Based Calibration (Proposed)

Rather than using min-max scaling, we estimate activation scale using a high-percentile threshold:

$$s = \frac{\text{percentile}_p(|x|)}{2^{b-1} - 1} \tag{19}$$

with default $p = 99.9$.

This reduces the influence of extreme outliers while preserving a uniform global scale.

Unlike PEG or mixed precision, percentile calibration:

- Requires no architectural modification
- Preserves single-scale per tensor
- Adds no runtime cost
- Maintains compatibility with integer-only inference

The tradeoff is potential clipping of rare extreme values; however, we empirically observe that clipping a small fraction of activations significantly improves effective resolution for the majority of values.

E. Implementation Consistency

All methods:

- Use identical calibration datasets
- Apply static quantization (no runtime range updates)
- Retain identical inference batch size and sequence length
- Serialize models in the same format for size comparison

This ensures that observed differences arise solely from quantization strategy rather than implementation artifacts.

V. Accuracy Results

Table I reports validation accuracy on QNLI for all quantization strategies. FP32 serves as the baseline reference.

TABLE I
VALIDATION ACCURACY ON QNLI

Method	Accuracy (%)	Δ vs FP32
FP32	**89.66**	–
W8A8	54.33	-35.33
Mixed Precision	89.42	-0.24
PEG (K=3,P)	66.12	-23.54
Percentile	50.54	-39.12

A. Observations

Several important patterns emerge:

- **Naive W8A8 collapse.** Uniform INT8 activation quantization reduces accuracy by 35.33 points, confirming severe degradation due to activation range distortion.
- **Mixed precision robustness.** Retaining sensitive layers in FP16 nearly restores full accuracy (only -0.24), indicating that specific layers dominate quantization sensitivity.
- **PEG partial recovery.** Per-embedding-group scaling improves over naive W8A8 but still suffers a 23.54 point drop, suggesting grouping alone does not fully eliminate structured outlier effects.
- **Percentile instability.** Contrary to expectations, percentile calibration at $p = 99.9$ further degrades performance (-39.12). This indicates that aggressive clipping in this configuration removes semantically important activation mass rather than merely suppressing noise.

B. Implications

These results highlight that transformer activation quantization failure is not purely due to extreme rare outliers. Instead, dominant activation channels appear to encode semantically meaningful signal that cannot be naively clipped without harming model expressiveness.

Mixed precision succeeds because it avoids compressing these channels entirely. PEG improves resolution allocation but does not fully decouple structured dominant dimensions.

The percentile method, while computationally simple, demonstrates that clipping must be applied carefully; percentile thresholds that are too aggressive may discard essential information.

Overall, these results confirm that activation quantization sensitivity in transformers is structurally different from CNN behavior and requires channel-aware mitigation strategies.

VI. Deployment Profiling

We evaluate runtime and memory characteristics for all quantization strategies using batch size 8 and sequence length 128 on an NVIDIA RTX 3050 (6GB VRAM).

A. Latency Analysis

Contrary to expectations, INT8 quantization does not produce measurable latency improvements on RTX 3050 hardware. All methods cluster tightly around 58–59 ms at p50, with minimal variation in p95.

This behavior is attributable to several factors:

TABLE II
LATENCY, VRAM, AND SERIALIZED MODEL SIZE

Method	p50 (ms)	p95 (ms)	VRAM (MB)	Size (MB)
FP32	58.38	59.08	483.7	417.7
W8A8	58.61	59.13	485.5	418.3
MP-PTQ	58.77	59.55	486.3	418.3
PEG	58.97	59.81	486.3	419.6
Percentile	59.12	59.77	486.3	418.3

- PyTorch execution primarily dispatches to optimized FP32 CUDA kernels.
- The RTX 3050 does not provide strong Tensor Core acceleration benefits for small-batch INT8 transformer inference in this configuration.
- Kernel launch overhead dominates arithmetic savings at batch size 8.

Thus, under this deployment regime, arithmetic precision reduction does not translate into runtime improvement.

B. VRAM Usage

Peak VRAM usage remains effectively unchanged across methods (approximately 484–486 MB). This indicates that:

- Activation buffers dominate memory footprint at runtime.
- INT8 weight storage alone does not significantly reduce peak allocated CUDA memory.
- Framework-level tensor representations may still allocate in FP32 internally.

This result highlights an important practical insight: theoretical parameter compression does not necessarily reduce runtime memory footprint without specialized inference kernels.

C. Serialized Model Size

Serialized model size differences are minimal:

- FP32: 417.7 MB
- INT8 variants: 418–419 MB

The limited reduction arises because:

- Checkpoint serialization includes metadata.
- Certain tensors remain in higher precision.
- Storage format does not strictly enforce 4× reduction in all components.

Thus, while quantization conceptually reduces arithmetic precision, practical deployment benefits depend heavily on hardware and runtime implementation details.

D. Systems-Level Implications

These findings emphasize a key systems lesson:

> Quantization alone does not guarantee deployment speedup. Hardware support and kernel implementation determine realized gains.

On this GPU configuration, activation quantization primarily affects numerical behavior rather than runtime efficiency. Consequently, the tradeoff landscape must be evaluated jointly across accuracy and hardware characteristics.

TABLE III
DEPTH-WISE ACTIVATION OUTLIER STATISTICS (FP32)

Layer	Mean Variance	Kurtosis	Top-1% Energy
Embeddings (0)	0.25	9	0.15
Layer 1	0.32	14	0.21
Layer 2	0.38	41	0.29
Layer 4	0.46	73	0.37
Layer 10	0.55	135	0.49
Layer 11	0.58	271	0.55
Pooler (12)	0.54	73	0.53

For true latency improvements, hardware-aware quantization or dedicated INT8-optimized inference engines (e.g., TensorRT or ONNX Runtime with INT8 kernels) would likely be required.

VII. STATISTICAL OUTLIER ANALYSIS

To quantify activation outlier behavior, we analyze FP32 activations from a fine-tuned checkpoint using a 64-sample validation slice. For each transformer layer, we compute:

- Mean per-dimension variance
- Kurtosis (fourth standardized moment)
- Top-1% energy fraction (fraction of total squared activation energy contributed by top 1% largest-magnitude dimensions)

A. Depth-Wise Activation Statistics

Table III summarizes representative layers.

B. Variance Amplification

Mean activation variance approximately doubles from early layers (0.25) to deep layers (0.58). This confirms residual accumulation increases activation magnitude scale across depth.

C. Heavy-Tailed Behavior

Kurtosis increases dramatically with depth, reaching 271 at Layer 11. For reference, a Gaussian distribution has kurtosis of 3. Values exceeding 100 indicate extreme heavy-tailed behavior.

This confirms that transformer activations deviate strongly from light-tailed assumptions underlying uniform min-max quantization.

D. Structured Energy Concentration

The top-1% energy fraction grows from 15% at embeddings to 55% at Layer 11. This implies that:

$$\sum_{i \in \text{top-1\%}} x_i^2 \approx 0.55 \sum_j x_j^2 \tag{20}$$

Thus, a very small subset of embedding dimensions dominates activation magnitude.

This structured dominance explains why global min-max scaling is ineffective: the dynamic range is determined by a small number of extreme channels, compressing the remaining 99% of dimensions.

E. Connection to Quantization Collapse

Combining these findings:

- Variance grows with depth (residual amplification).
- Kurtosis increases sharply (heavy tails).
- Energy concentrates in a small subset of dimensions.

Under uniform affine scaling, scale is proportional to $\max(|x|)$, which is driven by these dominant channels. Consequently, the bulk of activation mass receives reduced effective resolution, increasing quantization error and producing the severe accuracy collapse observed in W8A8.

These statistics strongly support the hypothesis that transformer PTQ failure is driven by structured channel dominance rather than random noise.

VIII. Ablation Study

We conduct controlled ablations to isolate which components most influence quantization robustness.

A. Percentile Threshold Sensitivity

We vary the percentile threshold p in the proposed calibration method:

- $p = 99.0$
- $p = 99.5$
- $p = 99.9$
- $p = 99.99$

Across all thresholds, validation accuracy remains approximately 50.54% with loss ≈ 0.711. No measurable improvement occurs beyond 99.9 clipping.

Observation: Percentile clipping does not recover performance in this configuration.

Interpretation: Structured dominant channels likely encode semantically meaningful signal rather than rare noise. Clipping even 0.01% of extreme activations appears to remove important information rather than merely suppressing statistical outliers.

This reinforces the conclusion that transformer activation outliers are structured and functional rather than random anomalies.

B. PEG Grouping Strategy

We evaluate PEG with permutation across group counts $K \in \{2, 3, 4\}$. The results exhibit a sharp non-monotonic dependence on K:

- $K = 2$: 49.46% accuracy (worse than naive W8A8)
- $K = 3$: 66.12% accuracy (partial recovery)
- $K = 4$: 86.18% accuracy (near-FP32 behavior)

This suggests PEG effectiveness depends critically on whether grouping provides sufficient resolution to isolate dominant channels. With too few groups ($K = 2$), dominant dimensions still saturate group range estimation, collapsing the remaining dimensions. Increasing to $K = 4$ appears to allocate enough independent scale factors to preserve information in non-dominant channels while limiting outlier-driven range inflation.

C. Mixed Precision Layer Sensitivity

Mixed precision retains selected layers in FP16 and achieves 89.42% accuracy (-0.24 relative to FP32).

This suggests that quantization sensitivity is highly localized. Protecting feed-forward output projections and residual pathways is sufficient to preserve model expressiveness.

Combined with the statistical findings:

- Heavy-tailed activation distributions
- Depth-wise kurtosis explosion
- Concentrated channel energy

we infer that certain layers act as amplification bottlenecks for quantization error. Mixed precision succeeds by avoiding compression at these bottlenecks.

D. Accuracy–Efficiency Tradeoff

Table I and Table II together reveal:

- Mixed precision restores accuracy but does not improve latency.
- PEG partially recovers accuracy with minimal runtime overhead.
- Percentile calibration adds no runtime cost but fails to restore accuracy.
- INT8 arithmetic alone does not yield measurable speedup on RTX 3050 hardware.

Thus, mitigation strategies must be evaluated not only for statistical correction but also for hardware-aligned efficiency gains.

E. Key Insight from Ablations

The ablation results strongly suggest:

> Transformer PTQ failure is driven more by structured channel dominance than by rare extreme scalar outliers.

Simple percentile clipping is insufficient. Channel-aware strategies (mixed precision or more granular grouping) are required to preserve accuracy.

IX. Discussion

Key Findings:

- Naive W8A8 collapses under structured outliers.
- PEG restores accuracy with minor overhead.
- Mixed precision is robust but reduces compression.
- Percentile calibration offers strong middle ground.
- Deployment gains must be evaluated jointly with latency.

X. Limitations

- Limited to BERT-base scale.
- Single GPU hardware.
- No LLM-scale experiments.

XI. Future Work

- Scaling to LLaMA-style architectures.
- Hardware-aware quantization for NPUs.
- Formal analysis of residual amplification.

XII. Conclusion

We present a reproducible study of activation outlier-induced PTQ failure in transformers. Our percentile-based calibration provides a deployment-friendly alternative to more complex grouping strategies. Joint accuracy and profiling analysis clarifies practical tradeoffs for edge deployment.

Reproducibility Statement

Code and logs available at:

https://github.com/pranavkkp4/TransformerTheory

Includes:

- One-click runner
- Automatic aggregation
- Paper auto-generation
- Environment specification

References

[1] Y. Bondarenko, M. Nagel, and T. Blankevoort, "Understanding and overcoming the challenges of efficient transformer quantization," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[2] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[3] M. Nagel, R. A. Amjad, M. van Baalen, C. Louizos, and T. Blankevoort, "Data-free quantization through weight equalization and bias correction," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2020.

[4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Llm.int8(): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.